System Info

This document covers cross-cutting behavior, shared components, and infrastructure that span multiple pages of the resmon app. Page-specific behavior lives in the ten companion documents (`dashboard_info.md` , `deep_dive_info.md` , `deep_sweep_info.md` , `routines_info.md` , `calendar_info.md` , `results_and_logs_info.md` , `configs_info.md` , `monitor_info.md` , `repos_and_api_keys_info.md` , `settings_info.md`) and is referenced here rather than repeated.

System-Wide Overview

App Purpose

resmon (Research Monitor) is a local-first literature-surveillance desktop application that queries a curated catalog of open-access scholarly repositories, normalizes and deduplicates results across sources, persists them to a local SQLite database, optionally summarizes each paper via a user-configured BYOK LLM provider, and surfaces the resulting artifacts through an Electron + React UI. It supports on-demand manual runs ("Deep Dive" single-repo, "Deep Sweep" multi-repo) as well as cron-scheduled automated sweeps ("Routines") driven by a persistent APScheduler job store. Optional integrations include transactional email (SMTP), Google Drive artifact backup, and a closed-beta multi-user cloud mirror.

High-Level Architecture

The application is a three-process composition:

1. **Electron main process** — launches the Python backend as a child process, hosts the renderer window, bridges OS-level capabilities (directory chooser, shell open/reveal, cloud-auth keychain bridge via `window.resmonAPI` , see `frontend/src/api/client.ts`), and can attach to an already-running headless daemon instead of spawning its own backend (see `implementation_scripts/daemon.py`).
2. **Python FastAPI backend** (`resmon_scripts/resmon.py`) — exposes a local REST API on `127.0.0.1:<port>` (default `8742` , surfaced to the renderer via `window.resmonAPI.getBackendPort()`), owns the SQLite connection, orchestrates the sweep pipeline, hosts the APScheduler instance, and streams progress events over Server-Sent Events.
3. **Optional cloud service** — separate microservice (IMPL-27..40) backing the closed-beta multi-device sync surface; the desktop app consumes it through the `cloudClient` wrapper plus `useCloudSync` hook.

Frontend ↔ backend communication is exclusively `fetch` -over-localhost JSON plus SSE for progress. There is no IPC bridge between the renderer and Python; the Electron preload only exposes filesystem, shell, and cloud-auth helpers via a contextBridge-exposed `window.resmonAPI` surface.

Phase and Version Status

- `APP_NAME = "resmon"` , `APP_VERSION = "1.0.0"` (`implementation_scripts/config.py`).
- Phase 2 implementation is complete through IMPL-1..40, IMPL-R1..R12 (routine wiring + concurrent-execution monitor), and IMPL-AI1..AI14 (AI summarization wiring + provider expansion). This info-documentation tranche (IMPL-INFO1..INFO11) closes Phase 2; Phase 3 (Conclusion) is entered only under explicit user authorization.
- Schema version is tracked by `SCHEMA_VERSION` / `get_schema_version` in `implementation_scripts/database.py` .

Operational Modes

The app supports three operational modes that are orthogonal to the page surface:

1. **Local-only** — default. All routines, executions, and credentials live in the user's local SQLite database and OS keyring; cloud auth is not required.
2. **Hybrid** — local execution plus opt-in Google Drive artifact backup (see `Settings → Cloud Storage` panel and `settings_info.md`) and/or opt-in multi-device cloud sync (ingests remote `cloud_routines` / `cloud_executions` mirror rows via `useCloudSync`).
3. **Cloud-executed routines** (IMPL-37) — individual routines may be flagged `execution_location = "cloud"` so they run against the cloud service's worker path; local mirrors are pulled back on sync.

The Electron shell additionally supports a **headless daemon split** (IMPL-25): the backend can run as a long-lived OS service (launchd on macOS, `systemd --user` on Linux, Task Scheduler on Windows; see `implementation_scripts/service_manager.py`), and multiple renderer windows can attach to that single daemon via the lock file written by `daemon.py` .

Artifact Topology

All user-facing outputs land under `resmon_reports/` at the project root (`REPORTS_DIR` in `config.py`):

- `resmon_reports/markdowns/` — Markdown report per execution (authored by `report_generator.save_report`).
- `resmon_reports/pdfs/` — optional PDF exports (when the user exports a report; see `results_and_logs_info.md`).
- `resmon_reports/figures/` — static figures included in reports.
- `resmon_reports/latex/` — LaTeX renditions of reports.
- `resmon_reports/logs/` — per-execution logs emitted by `TaskLogger` plus the rotating app log (`resmon.log` , 5 MB × 3 backups; `logger.setup_app_logger`).
- `resmon_reports/info_docs/` — the 11 implementation-grounded info documents produced by this tranche.
- `resmon_experiments/` and `resmon_printouts/` — sibling folders used for ad-hoc experimental artifacts outside the report pipeline.

The SQLite database file itself lives at `DEFAULT_DB_PATH = PROJECT_ROOT / "resmon.db"` , and the OS-scoped state directory (lock file, daemon logs, service-unit-managed logs) is resolved by `daemon.state_dir()` (`~/Library/Application Support/resmon` on macOS, `$XDG_STATE_HOME/resmon` on Linux, `%LOCALAPPDATA%\resmon` on Windows).

Frontend

Electron Shell and React Runtime

The desktop surface is an Electron shell hosting a React 18 + TypeScript single-page application bootstrapped from `frontend/src/index.tsx` and rooted at `App.tsx` . The Electron main process exposes a narrow API surface to the renderer through a contextBridge-injected `window.resmonAPI` object (typed in `api/client.ts`) whose members include `getBackendPort` , `platform` , `versions` , `chooseDirectory` , `openPath` , `revealPath` , and the `cloudAuth` bridge used by `AuthContext` . The renderer never invokes Python directly — it always goes through the REST client.

Webview Embed Hardening (Update 3)

The About resmon → Blog tab embeds the resmon GitHub Pages blog inside an Electron `<webview>` element. To support this safely:

- The renderer's `webPreferences` enables `webviewTag: true` (off by default since Electron 5). Every other security flag is held at its safe default: `nodeIntegration: false` , `contextIsolation: true` , `sandbox: true` for the renderer, and a per-window CSP (see below).
- The main process registers a `will-attach-webview` handler that mutates `webPreferences` on every `<webview>` instance before attachment. The handler **deletes** `nodeIntegration` , `nodeIntegrationInWorker` , `nodeIntegrationInSubFrames` , and `preload` , sets `contextIsolation: true` and `sandbox: true` , and rejects any `params.src` whose scheme is not `https:` or whose origin is not `https://ryanjosephkamp.github.io` . This guarantees an attached `<webview>` can never reach the host's IPC bridge or load arbitrary remote origins, even if the renderer-side React component is tampered with.
- The renderer's CSP `<meta>` tag in `frontend/public/index.html` is widened to allow the GitHub Pages origin: `connect-src 'self' http://127.0.0.1:8742 https://ryanjosephkamp.github.io;` (used by the Atom-feed fetch) and `frame-src https://ryanjosephkamp.github.io; child-src https://ryanjosephkamp.github.io;` (used by the embedded webview). All other CSP directives remain unchanged. No additional origins are allowlisted.
- The Blog component's `<webview>` wires `new-window` and `will-navigate` events to `window.resmonAPI.openPath` , which routes off-origin URLs to the user's default browser via `shell.openExternal` rather than navigating the embed. This keeps the embed origin-locked even when a blog post links to an external site.

Public Blog Source (`docs/` , GitHub Pages, Update 3)

The blog rendered inside the About resmon → Blog tab is served from the repository's `docs/` folder via GitHub Pages (Settings → Pages → Source: `main` / `docs`). Topology:

- `docs/_config.yml` — Jekyll site config: `title: "resmon Blog"` , `baseurl: /resmon` , `url: https://ryanjosephkamp.github.io` , `theme: minima` , and `plugins: [jekyll-feed]` to auto-generate `feed.xml` .
- `docs/Gemfile` — Jekyll + theme + plugin pins so the feed can be reproduced locally.
- `docs/index.md` — landing page; lists posts via `site.posts` .
- `docs/_posts/YYYY-MM-DD-<slug>.md` — one Markdown file per blog post; the front matter includes `layout: post` , `title:` , and `date:` .
- `docs/README.md` — operator notes for adding a post.

The Atom feed lives at `https://ryanjosephkamp.github.io/resmon/feed.xml` (auto-emitted by `jekyll-feed`) and is the URL `BlogTab` fetches and parses on mount.

GitHub Issue-Form Templates (`.github/ISSUE_TEMPLATE/` , Update 3)

The About resmon → Issues tab's "File on GitHub" submit path opens a deep link to a typed GitHub issue form so the user does not need to write Markdown by hand. The matching templates live at the repository root under `.github/ISSUE_TEMPLATE/` :

- `bug.yml` — `Bug report` form (fields: `description` , `steps_to_reproduce` , `expected` , `actual` , `version` , `os`).
- `feature.yml` — `Feature request` form (fields: `problem` , `proposed_solution` , `alternatives`).
- `question.yml` — `Question` form (fields: `question` , `context`).
- `config.yml` — disables blank issues; adds `contact_links` for general inquiries (`mailto:`) and the resmon Discussions tab.

The Issues tab encodes the user's form contents into the GitHub `issues/new?template=<file>.yml&title=<title>&description=<body>...` query string so the GitHub form auto-selects and pre-fills on the user's behalf.

HashRouter and Route Map

Routing is centralized in `App.tsx` via `react-router-dom` `HashRouter` . The route map is:

Path	Component	Info doc
`/`	`DashboardPage`	dashboard_info.md
`/dive`	`DeepDivePage`	deep_dive_info.md
`/sweep`	`DeepSweepPage`	deep_sweep_info.md
`/routines`	`RoutinesPage`	routines_info.md
`/calendar`	`CalendarPage`	calendar_info.md
`/results`	`ResultsPage`	results_and_logs_info.md
`/configurations`	`ConfigurationsPage`	configs_info.md
`/monitor`	`MonitorPage`	monitor_info.md
`/repositories`	`RepositoriesPage`	repos_and_api_keys_info.md
`/settings/*`	`SettingsPage` (nested router)	settings_info.md
`/about`	`AboutResmonPage` (Tutorials / Issues / Blog / About App tabs)	about_resmon_info.md

`HashRouter` is used rather than `BrowserRouter` because the renderer is loaded from `file://` in the packaged Electron app, where push-state history is unreliable. The `Sidebar` , `Header` , and `MainContent` layout components wrap every route; the `FloatingWidget` monitor overlay sits outside `MainContent` so it survives route transitions.

Shared Contexts

Two React contexts wrap the entire route tree (see `App.tsx`):

- **`AuthProvider`** (`context/AuthContext.tsx`) — exposes `{ isSignedIn, email, accessToken, signIn, signOut, refreshAccessToken }` for the resmon-cloud account (IMPL-30). Access tokens are held only in the module-scoped `authStore` — never in React state, `localStorage` , or disk. Refresh tokens live in the OS keychain, managed by Electron-main via the Python `keyring` bridge (service `resmon` , account `cloud_refresh_token`). `AuthContext` also tracks `sync_state` reported by the Electron bridge.
- **`ExecutionProvider`** (`context/ExecutionContext.tsx`) — global store for every concurrently-running execution. Keyed by execution id, it tracks `{ executionId, executionType: 'deep_dive' | 'deep_sweep' | 'automated_sweep', repositories, startTime, events, status, currentRepo` . It owns the SSE subscriptions, drives the FloatingWidget pulse, raises browser Notifications based on `notify_manual` / `notify_automatic_mode` (see `settings_info.md` → Notifications Panel), and exposes a `verboseLogging` toggle persisted under `localStorage["resmon.verboseLogging"]` . The multi-execution rewrite (IMPL-R8) replaced the single-execution store with `Record<number, ActiveExecution>` plus a focused-execution pointer that the FloatingWidget and `MonitorPage` tab strip read from.

Shared Hooks

Three cross-cutting hooks live in `frontend/src/hooks/` :

- **`useExecutionsMerged`** — fetches `/api/executions/merged` , which returns local SQLite executions plus cloud-mirror rows sorted by `start_time` DESC. Each row carries `execution_location ∈ {"local", "cloud"}` so `ResultsPage` can render the Local/Cloud badge and filter chip without inspecting id shapes. Auto-refreshes on: explicit `refresh()` , any `completionCounter` change from `ExecutionContext` , and the `resmon:cloud-sync-applied` window event emitted by `useCloudSync` .
- **`useRepoCatalog`** — loads `{catalog, bySlug, presence}` in parallel from the repository-catalog and credential-presence endpoints and exposes `refreshPresence()` . Used by the Deep Dive, Deep Sweep, Routines, and Repositories pages for a single source of truth on repository metadata and credential presence.
- **`useCloudSync`** — when the user is signed in and the window is focused, polls `GET /api/v2/sync?since=<last_synced_version>` on a 60-second interval, POSTs returned pages to the local daemon's `/api/cloud-sync/ingest` , drains `has_more` chains within a tick (cap `MAX_PAGES_PER_TICK = 50`), and triggers an immediate sync on focus-gain. On sign-out, it calls `/api/cloud-sync/clear` once. All errors are surfaced via `state.lastError` rather than propagated, per the "transient cloud outage must not crash the renderer" invariant.

API-Client Wrappers

Four API-client modules centralize all HTTP access:

- **`api/client.ts`** — default wrapper for the local FastAPI daemon. Resolves the base URL to `http://127.0.0.1:<port>` where the port comes from the Electron-injected `window.resmonAPI.getBackendPort()` (default `8742`). Every request sets `Cache-Control: no-store` and `Pragma: no-cache` . On non-2xx responses, the wrapper unwraps FastAPI's `{detail: "..."}` envelope and re-throws with a human-readable `"${status} ${statusText}: ${detail}"` message.
- **`api/cloudClient.ts`** — wrapper for the remote cloud service. Exports `CloudUnauthorizedError` so `useCloudSync` can distinguish token-expiry from transport errors.
- **`api/repositories.ts`** — typed helpers over the repository-catalog and credential-presence endpoints, consumed by `useRepoCatalog` .
- **`api/authStore.ts`** — module-scoped access-token store with refresh registration and a sign-out broadcaster.

FloatingWidget and Monitor Integration

The `FloatingWidget` (`components/Monitor/FloatingWidget.tsx`) is mounted once at the App root and renders an overlay that summarizes the focused execution plus a stack popover for any additional concurrent executions (IMPL-R10). It is driven entirely by `ExecutionContext` and reacts to `location.pathname` changes: it auto-minimizes on `/monitor` , any `/results` , and `/calendar` , since those pages already surface the full execution state on-screen. It supports cancel, clear, and focus-switch actions, and pulses on completion until the user acknowledges. The dedicated `/monitor` page renders the same data in a tab-strip + detail-pane layout; see monitor_info.md.

Theming and Badge Palette Conventions

Global stylesheet lives in `frontend/src/styles/` and is imported from `index.tsx` . Conventional class families used across pages:

- Status pill classes: `status-running` , `status-completed` , `status-failed` , `status-cancelled` , `status-cancelling` , `status-pending` .
- Repository-status glyphs in the widget/monitor: ✓ (done), ⟳ (querying), ✗ (error), ○ (pending) (`FloatingWidget.repoIcon`).
- Execution-type labels: `"Deep Dive"` , `"Deep Sweep"` , `"Auto Sweep"` (`FloatingWidget.typeLabel`).
- Stage labels used in the widget + Monitor page: `querying` → "Querying repositories", `dedup` → "Deduplicating results", `linking` → "Linking documents", `reporting` → "Generating report", `summarizing` → "AI summarization", `finalizing` → "Finalizing".
- The Results page adds a Local/Cloud origin badge driven by the `execution_location` field on merged rows.

Per-panel navigation links use the `tab-btn` / `tab-active` pair; see `settings_info.md` for the Settings tab-strip contract.

Cross-Page References

Cross-page flows that do not belong to a single info doc:

- Clicking a **Configuration** row on `/configurations` pre-fills the query form on `/dive` or `/sweep` (see `configs_info.md`).
- Completing an execution on `/dive` , `/sweep` , or the Routines "Run Now" action auto-navigates (or offers navigation) to `/results?execution_id=…` once the SSE stream closes (see `deep_dive_info.md` , `deep_sweep_info.md` , `routines_info.md`).
- The `/calendar` page derives its events from the same `/api/executions/merged` stream that feeds `/results` , keyed by `start_time` (`calendar_info.md`).

- The **credential presence map** exposed by `/api/credentials` is consumed by the Settings → Email/AI panels, the Repositories page, and the inline "Use a key just for this run" affordance on the Dive/Sweep/Routines forms.

In-Renderer Pub/Sub Buses

Several pages share state that is owned by a single source-of-truth page but consumed elsewhere. To keep cross-page edits propagated without forcing manual reloads, the renderer ships small in-process pub/sub buses under `resmon_scripts/frontend/src/lib/` :

- `configurationsBus.ts` — exports `notifyConfigurationsChanged()` and `useConfigurationsVersion()` . Bumped whenever a saved configuration (Dive query template, Sweep query template, or Routine) is created, edited, deleted, or migrated. Subscribers: `ConfigurationsPage` , `ConfigLoader` instances on Dive / Sweep / Routines, and any future surface that reads `/api/configurations` .
- `routinesBus.ts` — exports `notifyRoutinesChanged()` and `useRoutinesVersion()` . Mirrors the configurations bus pattern but is scoped to routine-shaped state. Bumped whenever a routine is created, edited, deleted, activated, deactivated, or migrated. Subscribers: `RoutinesPage` (its local + cloud fetch effects) and `CalendarPage` (its `fetchData` effect, so the cron-expanded fire times update in place after a save from the popover).
- **Dual-broadcast contract:** the shared `RoutineEditModal` (used by both `RoutinesPage` and the `CalendarPage` event popover's `Edit Routine` button) fires BOTH buses on save, because a routine save also invalidates routine-typed configuration views.

Backend

FastAPI Application Shell

`resmon.py` constructs a single FastAPI application at module load: `app = FastAPI(title=APP_NAME, version=APP_VERSION)` . A single shared `sqlite3.Connection` (`_shared_conn`) is reused for every request via `_get_db()` / `_close_db()` ; the DB path is overridable for tests via `_db_path` . `init_db` runs lazily on first access through `_db_initialized` . The database file defaults to `DEFAULT_DB_PATH` but tests can set `_db_path = ":memory:"` (with the scheduler jobstore redirected to a temp file so APScheduler's SingletonThreadPool can share schema).

Cross-Cutting Middleware

Two ASGI middlewares are registered, in order:

1. **`PrivateNetworkMiddleware`** — a raw ASGI middleware (not `BaseHTTPMiddleware` , because the latter buffers streaming responses). It injects the `Access-Control-Allow-Private-Network: true` header on every HTTP response start so Chromium's Private Network Access policy allows the `file://` renderer to reach `127.0.0.1` . SSE responses are unaffected because no buffering is introduced.
2. **`CORSMiddleware`** — `allow_origins=["*"]` , `allow_methods=["*"]` , `allow_headers=["*"]` . Safe because the backend binds to the loopback interface.

Admission Controller

The `admission` singleton (`implementation_scripts/admission.py` , `ExecutionAdmissionController` , ADQ-R3, IMPL-R1) is the single global gate on concurrent executions. Key invariants:

- Clamps `max_concurrent` to `[1, 8]` and `queue_limit` to `[1, 64]` ; defaults `DEFAULT_MAX_CONCURRENT = 3` , `DEFAULT_QUEUE_LIMIT = 16` .
- **Manual admission** is reject-or-pass: if no slot is free, the REST layer raises HTTP 429 (see Deep Dive / Deep Sweep / manual-search endpoints via IMPL-R2).
- **Routine admission** falls through to a bounded FIFO queue (`deque`) that drains as slots free. Overflow past `queue_limit` is dropped with a logged error (APScheduler worker threads must never raise).
- All state transitions are guarded by `_lock` . Queue drains spawn a fresh daemon thread per dispatch so the `finally` -path of a completing pipeline is never blocked by the next pipeline starting.
- Live values are reloaded from the `settings` table on startup (`_init_admission_on_startup` → `_hydrate_admission_from_db`) and updated via `PUT /api/settings/execution` (see `settings_info.md` → Advanced Panel and IMPL-R12).

SweepEngine Pipeline

`SweepEngine` (`implementation_scripts/sweep_engine.py`) orchestrates the complete query → normalize → dedup → report pipeline for both manual and routine-fired executions. The same code path is entered via `execute_dive` (single repo), `execute_sweep` (multi-repo), or `prepare_execution` + `run_prepared` (split pattern used when the REST layer needs the exec id before kicking off the background thread). Per-execution stages emit

`progress_store` events at `querying`, `dedup`, `linking`, `reporting`, `summarizing`, and `finalizing` — the same identifiers consumed by the FloatingWidget's `stageLabel` map.

Cross-cutting responsibilities:

- **Required-credentials gate** — the `_REQUIRED_CREDENTIALS` table maps repositories that cannot return any results without a key (`core`, `ieee`, `nasa_ads`, `springer`) to their credential name. Missing credentials short-circuit the repo with a user-visible error.
- **Cooperative cancellation** — `_search_with_heartbeat` polls `progress_store`'s cancel flag on a 2-second heartbeat; `_ExecutionCancelled` is caught by `_handle_cancellation` to persist a partial report.
- **Optional auto-backup** — `_maybe_auto_backup` invokes `cloud_storage.upload_directory` when the `cloud_auto_backup` setting is true and a Google Drive token is present.
- **Per-execution task log** — `TaskLogger` writes structured per-execution logs under `resmon_reports/logs/` (see `results_and_logs_info.md`).

ResmonScheduler

`ResmonScheduler` (`implementation_scripts/scheduler.py`) wraps APScheduler's `BackgroundScheduler` with a `SQLAlchemyJobStore` backed by `sqlite:///<DEFAULT_DB_PATH>` (ADQ-3). Jobs persist across app restarts. The module is deliberately decoupled from FastAPI and `SweepEngine` through a dispatcher indirection (IMPL-R3):

- `set_dispatcher(fn)` installs the callback invoked on each routine fire. `fn` receives `(routine_id: int, parameters: str)`.
- When no dispatcher is installed, `_routine_callback` logs an error and returns — the APScheduler worker thread never raises.
- Cron parsing is strict 5-field (`minute hour day month day_of_week`) via `_parse_cron`.

The FastAPI startup hook (IMPL-R4) calls `set_dispatcher(_dispatch_routine_fire)`, instantiates a single `ResmonScheduler`, starts it, then walks `get_routines(conn)` and re-registers every active routine so jobs survive a cold restart. The shutdown hook gracefully stops the scheduler and clears the dispatcher. `_dispatch_routine_fire` consults the admission controller before spawning a pipeline, stamps `executions.routine_id`, launches via `_launch_execution`, and updates `routines.last_executed_at` on completion. See routines_info.md for the full fire lifecycle and settings_info.md → Advanced Panel for the `/api/scheduler/jobs` diagnostics surface.

ProgressStore and SSE Progress Stream

`ProgressStore` (`implementation_scripts/progress.py`) is the thread-safe in-memory bus between the pipeline worker threads and the SSE endpoint. Each `exec_id` owns its own event list, `threading.Lock`, `threading.Event` cancel flag, and completion boolean. The API is `register(exec_id)`, `emit(exec_id, event)`, `get_events(exec_id, since)`, `mark_complete(exec_id)`, `cleanup(exec_id)`. The SSE endpoint pages through `get_events(exec_id, since=<cursor>)` until `_completed` flips; post-completion the REST layer persists the buffered events into the `execution_progress` table via `save_progress_events` so `ReportViewer` can reconstruct the stream after the process restarts (IMPL-21). See monitor_info.md and results_and_logs_info.md.

Cloud Integration

Two distinct cloud surfaces exist and must not be conflated:

1. **Google Drive artifact backup** (`implementation_scripts/cloud_storage.py`) — least-privilege `drive.file` OAuth 2.0 scope, `InstalledAppFlow.from_client_secrets_file` using `credentials.json` at the project root, token serialized and stored in the OS keyring under `google_drive_token`. Public helpers: `check_connection`, `authorize_google_drive`, `revoke_authorization`, `upload_directory`, `is_token_stored`, `probe_api`. Surfaced through `Settings → Cloud Storage` (see settings_info.md).
2. **resmon-cloud account + multi-device sync** (IMPL-27..40) — separate microservice backing the closed-beta experience. Desktop consumes it via `AuthContext`, `api/cloudClient.ts`, `useCloudSync`, and the local-daemon ingest endpoint (`/api/cloud-sync/ingest`, `/api/cloud-sync/clear`). Envelope-encrypted per-user credentials (ADQ-11, IMPL-31): per-user DEK wrapped by a KMS-held KEK. JWKS-verified JWTs, per-user row-level security (IMPL-28..29), signed-URL artifact fetches (IMPL-34), per-user rate limiting (IMPL-33). The `Settings → Cloud Account` panel is currently "under construction" — backend endpoints exist but the UI does not wire up a hosted IdP in this build (see settings_info.md).

Credential Manager and Ephemeral Key Stack

`implementation_scripts/credential_manager.py` is the only module that touches the OS keyring (ADQ-4). It defines two frozensets used as whitelists by `PUT /api/credentials/{name}`:

- `AI_CREDENTIAL_NAMES` — `openai_api_key`, `anthropic_api_key`, `google_api_key`, `xai_api_key`, `meta_api_key`, `deepseek_api_key`, `alibaba_api_key`, `custom_llm_api_key` (ADQ-AI9).

- `SMTP_CREDENTIAL_NAMES` — `smtp_password` .

Additional repository credentials (`core_api_key` , `ieee_api_key` , `nasa_ads_api_key` , `springer_api_key` , plus any optional/recommended keys from the catalog) are whitelisted via `repo_catalog.credential_names` . Core API: `store_credential` , `get_credential` , `delete_credential` , `validate_api_key` . The ephemeral-key stack (`push_ephemeral` , `pop_ephemeral` , `get_credential_for`) lets Deep Dive / Deep Sweep / Routines pass a per-execution key that lives only in-process, is keyed by `exec_id` , is never logged, and takes precedence over the persisted keyring value (IMPL-23). Credential values are never returned by any GET endpoint; only presence booleans are exposed through `/api/credentials` .

LLM Factory and Provider Whitelist

`implementation_scripts/llm_factory.py` (`build_llm_client_from_settings`) turns the persisted `ai_*` settings into a concrete `RemoteLLMClient` or `LocalLLMClient` , or returns `None` when the provider is unset or its credentials are missing (ADQ-AI7 / F6 — "AI unconfigured" is a silent no-op, never an exception). The only `ValueError` the factory raises is for `ai_provider == "custom"` with an insecure `ai_custom_base_url` : `_validate_custom_base_url` rejects non-HTTPS URLs unless the host is a loopback (`localhost` , `127.0.0.1` , `::1`). API keys are never logged, never included in exception messages, and never returned by any public function (ADQ-AI8; OWASP A02).

Key lookup order per ADQ-AI9: ephemeral `{provider}_api_key` → ephemeral `custom_llm_api_key` (custom provider only) → keyring `{provider}_api_key` → keyring `custom_llm_api_key` (custom provider only). The Settings → AI panel drives the same whitelist (IMPL-AI9 / AI10), the custom-URL guard is enforced both in the UI (IMPL-AI12) and in this factory, and `implementation_scripts/ai_models.list_available_models` (surfaced through the AI panel's "Load models" button) uses `httpx` to fetch the provider's model index without logging credentials. See settings_info.md → AI Panel.

Database Schema and Cloud-Mirror Cache

The SQLite schema is owned by `implementation_scripts/database.py` (ADQ-5); all queries are parameterized. Core tables referenced across pages (not exhaustive):

- `routines` — user-defined automation; `execution_location ∈ {"local","cloud"}` (IMPL-37).
- `executions` — every manual or routine-fired run; `routine_id` stamped by the dispatcher.
- `execution_progress` — persisted progress events (IMPL-21).
- `documents` + cross-tables — deduplicated paper records and per-execution links.
- `configurations` — saved query presets (`configs_info.md`).
- `settings` — flat key/value settings table accessed via `get_setting` / `set_setting` .
- `cloud_routines` / `cloud_executions` — cursor-synced read-only mirror of the cloud service's rows (IMPL-35..36).
- `sync_state` — holds `last_synced_version` per `get_last_synced_version` / `set_last_synced_version` .
- Cloud-cache bookkeeping — `record_cloud_cache_entry` , `touch_cloud_cache_entry` , `get_cloud_cache_entry` , `get_cloud_cache_total_bytes` , `evict_cloud_cache_if_needed` , with `CLOUD_CACHE_MAX_BYTES_DEFAULT` as the LRU bound.

Schema-version tracking (`SCHEMA_VERSION` , `get_schema_version`) runs migrations on startup via `init_db` .

Settings Surface Summary

Every persisted preference is eventually a row in the `settings` table. The Settings page is divided into eight panels (see settings_info.md): **Email**, **Cloud Account**, **Cloud Storage**, **AI**, **Storage**, **Notifications**, **Advanced**, **About App**. Cross-cutting behaviors reached from multiple pages:

- `notify_manual` / `notify_automatic_mode` — consumed by `ExecutionContext.maybeNotifyCompletion` .
- `cloud_auto_backup` — consumed by `SweepEngine._maybe_auto_backup` .
- `max_concurrent_executions` / `routine_fire_queue_limit` (IMPL-R12) — consumed by the admission controller.
- `ai_provider` / `ai_model` / `ai_local_model` / `ai_custom_base_url` / `ai_custom_header_prefix` / `ai_summary_length` / `ai_tone` / `ai_temperature` / `ai_extraction_goals` — consumed by `llm_factory.build_llm_client_from_settings` and the summarizer.
- `export_directory` — consumed by report export actions on the Results page.
- `pdf_policy` / `txt_policy` / `archive_after_days` — reserved for a future per-paper artifact feature; currently no effect on the sweep pipeline (documented as such in `settings_info.md`).

Logging Topology

Three logging surfaces coexist:

1. **Rotating app log** — `logger.setup_app_logger(log_dir)` creates a `RotatingFileHandler` at `resmon_reports/logs/resmon.log` (5 MB × 3 backups, UTF-8). Format: `%(asctime)s [%(levelname)s] %(name)s: %(message)s` . Every module uses `logging.getLogger(__name__)` so log

lines are traceable to their source.

2. **Per-execution task log** — `TaskLogger` writes a structured log per execution id into `resmon_reports/logs/` and is linked from `ReportViewer` (see results_and_logs_info.md).
3. **In-memory progress events** — `progress_store` (see above) — transient, then persisted to `execution_progress` on completion.

The renderer-side `verboseLogging` toggle (stored under `localStorage["resmon.verboseLogging"]`) controls only the client-side console verbosity of `ExecutionContext` ; it does not alter backend log levels.

Report Artifact Layout

Reports generated by `report_generator.generate_report` + `save_report` land under the directories enumerated in "Artifact Topology" above. File naming is deterministic per execution (stamp + execution id + execution type — see `resmon_implementation_guide.md` Appendix D for the canonical contract). Exports (PDF, LaTeX) are triggered from `ResultsPage` via explicit user actions; the Markdown report is always produced. See results_and_logs_info.md for the per-report viewer UI and for how `report_exporter.py` is invoked.

Cross-Page References

Cross-cutting backend flows that touch more than one page:

- **Notifications pipeline** — `GET/PUT /api/settings/notifications` is owned by the Settings → Notifications panel, but the actual notification firing happens inside `ExecutionContext.maybeNotifyCompletion` across every page that runs executions.
- **Execution-settings admission wiring** — `GET/PUT /api/settings/execution` lives in the Settings → Advanced panel (IMPL-R12) but binds the singleton used by Deep Dive, Deep Sweep, and routine dispatch.
- **Credential presence map** — `GET /api/credentials` returns a single `{name: bool}` envelope consumed simultaneously by `useRepoCatalog` , `EmailSettings` , `AISettings` , and the Repositories page's scope selector (IMPL-38, keyring-vs-cloud).
- **Merged executions feed** — `/api/executions/merged` is the sole backend source for both the Results page list and the Calendar page event stream.
- **Startup reconciliation** — `_init_admission_on_startup` and `_init_scheduler_on_startup` together guarantee that the admission cap and active-routine registrations match the persisted DB state before any request is served.

System Info

System-Wide Overview

App Purpose

High-Level Architecture

Phase and Version Status

Operational Modes

Artifact Topology

Frontend

Electron Shell and React Runtime

HashRouter and Route Map

Shared Contexts

Shared Hooks

API-Client Wrappers

FloatingWidget and Monitor Integration

Theming and Badge Palette Conventions

Cross-Page References

Backend

FastAPI Application Shell

Cross-Cutting Middleware

Admission Controller

SweepEngine Pipeline

ResmonScheduler

ProgressStore and SSE Progress Stream

Cloud Integration

Credential Manager and Ephemeral Key Stack

LLM Factory and Provider Whitelist

Database Schema and Cloud-Mirror Cache

Settings Surface Summary

Logging Topology

Report Artifact Layout

Cross-Page References